SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I M.Gnanesh Kumar Naidu / 192371051 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

Question 1

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

Answer:

1.1 Introduction

An **intelligent agent** is anything that perceives its environment through sensors and acts on it through actuators so as to achieve its objectives. The agent function maps the percept history to an action. The smart home is a good example: its sensors are motion detectors, temperature and light sensors, door/window contacts and the clock; its actuators are lights, heating/cooling (HVAC), door locks, alarms and phone notifications. Its performance measure combines three aims: **comfort, energy efficiency and security**. The environment is partially observable (the system cannot see who is in every room or what the residents intend), stochastic (people behave unpredictably), sequential, dynamic and continuous.

1.2 Types of Intelligent Agents and Their Role in the Smart Home

(a) Simple Reflex Agent

A simple reflex agent selects an action using only the **current percept**, through condition–action rules. It keeps no memory and no model of the world.

- **Smart-home functioning:** IF motion detected AND room dark THEN switch on light. IF temperature > 28 °C THEN switch on the air conditioner. IF door opened while the alarm is armed THEN sound the alarm.
- **Strengths:** very fast, simple, cheap and predictable; ideal for emergency and safety responses.
- **Weaknesses:** works only if the environment is fully observable from the current percept. It cannot tell whether a room is occupied once the person stops moving (the lights switch off while someone reads), cannot use time-of-day trends or past behaviour, and has no notion of energy cost or user preference.

(b) Model-Based Reflex Agent

This agent maintains an **internal state**, a model of how the world evolves and how its actions affect it, so it can handle partial observability.

- **Smart-home functioning:** it remembers that someone entered the living room 10 minutes ago and has not left, so it keeps the lights on even with no current motion; it tracks which windows are open, which rooms are occupied and the current heating state, and estimates how fast a room cools.
- **Strengths:** copes with partial observability and sensor gaps; avoids needless switching.
- **Weaknesses:** still follows fixed rules; it does not know what the user is trying to achieve and cannot plan ahead.

(c) Goal-Based Agent

A goal-based agent has an explicit **goal** and uses search or planning to choose actions that achieve it.

- **Smart-home functioning:** goal "living room at 22 °C when the owner arrives at 6 pm". The agent estimates the heating time required and starts pre-heating at 5:30 pm; goal "house secured by 11 pm" triggers a plan to check and lock all doors and windows and switch off appliances.
- **Strengths:** flexible, plans multi-step actions and adapts when the goal changes.
- **Weaknesses:** goals are binary (achieved or not). It cannot compare two ways of achieving the same goal, for example reaching 22 °C quickly at high energy cost against slowly at low cost.

(d) Utility-Based Agent

A utility-based agent assigns a **utility value** to each possible state and chooses the action with the highest expected utility, so it can weigh competing objectives and uncertainty.

- **Smart-home functioning:** $U = w_1 \cdot \text{comfort} - w_2 \cdot \text{energy cost} - w_3 \cdot \text{security risk}$. If electricity prices peak, the agent may allow the temperature to drift by 1 °C to save energy, but never lowers security. It can also decide that dimming the lights at 60% gives nearly the same comfort for 40% less energy.
- **Strengths:** handles trade-offs and uncertainty, and personalises behaviour through user-specific weights.
- **Weaknesses:** defining and tuning the utility function is hard; evaluating expected utility is more computational.

1.3 Comparison

Feature	Simple reflex	Model-based	Goal-based	Utility-based
Uses memory / model	No	Yes	Yes	Yes
Looks ahead / plans	No	No	Yes	Yes
Handles trade-offs	No	No	Limited	Yes
Speed of response	Fastest	Fast	Slower	Slowest
Smart-home example	Light on when motion	Lights stay on while room occupied	Pre-heat before arrival	Balance comfort vs energy cost
Best suited to	Safety alarms	Occupancy tracking	Scheduling	Optimising comfort/energy

1.4 Most Effective Approach: A Hybrid (Layered) Agent

No single agent type is enough, because the three objectives need different kinds of behaviour. Security and safety need **instant, reliable reactions**; comfort needs **memory of the house state and of the user's routine**; energy efficiency needs **planning and trade-offs**; and personalisation needs **learning**. The most effective design is therefore a **hybrid, layered architecture with a learning component**:

- **Layer 1 – Simple reflex (security and safety):** intruder, smoke or gas alarms, and door-locked checks run at the highest priority with no delay, and these rules override lower layers.
- **Layer 2 – Model-based (state tracking):** maintains occupancy, door/window status, temperature trends and a model of user habits, filling the gaps left by partial observability.
- **Layer 3 – Goal-based (planning):** schedules pre-heating, pre-cooling, night-time lock-up and "away mode" to achieve targets such as "comfortable on arrival".
- **Layer 4 – Utility-based (optimisation):** chooses among the possible plans by weighing comfort, energy cost and risk, including tariff periods and user preferences.
- **Learning component:** observes manual overrides (for example, the user lowering the temperature at night) and updates the habit model and the utility weights over time.

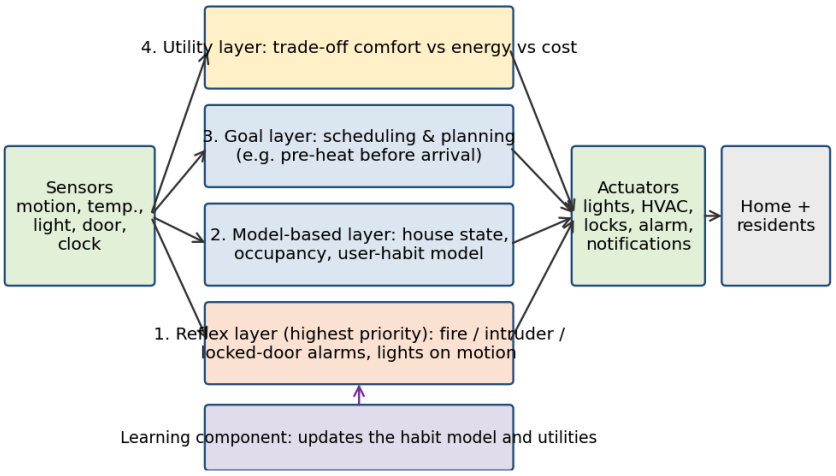


Figure 1: Hybrid layered smart-home agent

1.5 Example Scenario

It is 7 pm in winter and nobody is in the living room. The reflex layer checks that doors are locked and the alarm is armed. The model layer knows that the family usually returns at 7:30 pm. The goal layer sets the target "22 °C at 7:30 pm", and the utility layer chooses a heating schedule that meets the target with the lowest cost under the evening tariff. The learning component notes that the owner lowered the temperature to 21 °C last night and adjusts the target. Suddenly the front door opens at 2 am while the alarm is armed: the reflex layer fires the alarm immediately, without waiting for the higher layers.

1.6 Conclusion

Simple reflex agents give speed and dependable security, model-based agents give situational awareness, goal-based agents give planning, and utility-based agents give the trade-off between comfort and energy. A hybrid agent that combines all four with learning gives the best balance of **comfort, energy efficiency and security** in a smart home, because each layer covers the weaknesses of the others.

Question 2

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Answer:

2.1 Problem Understanding and Formulation

The robot must travel from its start cell to the exit of a maze whose layout is initially unknown. Because it has no heuristic information about where the exit lies, **uninformed (blind) search** is appropriate. The maze is formulated as a search problem:

- **States:** the robot's cell (and, as it learns, the map of known cells).
- **Initial state:** the start cell. **Goal test:** the robot is at the exit.
- **Actions:** move Up, Down, Left, Right into a cell that is not a wall.
- **Step cost:** 1 per move, or a different cost when cells differ (for example slopes, narrow corridors or rough floors).

Since the layout is unknown, the robot discovers walls and openings as it moves. The branching factor b is at most 3 after the first step (the cell it came from is not re-entered), and the depth d of the shallowest exit is unknown.

2.2 Uniform Cost Search (UCS)

UCS expands the frontier node with the **lowest path cost g(n)**, using a priority queue. It explores the maze in order of increasing cost, so the first time the exit is selected for expansion the path is the **cheapest**.

- **Use in the maze:** as the robot senses new cells it inserts them into the priority queue with their accumulated costs g(n); the cheapest unexplored cell is always visited next. With non-uniform costs (stairs, rough floor, risk) UCS finds the lowest-cost route, not just the shortest.
- **Properties:** complete if every step cost $\geq \varepsilon > 0$; optimal. When all costs are equal, UCS behaves like Breadth-First Search.
- **Time complexity:** $O(b^{(1 + \lfloor C^*/\varepsilon \rfloor)})$, where C^* is the optimal cost and ε the minimum step cost.
- **Space complexity:** $O(b^{(1 + \lfloor C^*/\varepsilon \rfloor)})$, because every frontier node must be stored.
- **Advantages:** guarantees the least-cost path; handles variable costs; complete.

- **Disadvantages:** explores in all directions (it has no sense of direction toward the exit); uses a great deal of memory in large mazes; many cheap steps in the wrong direction can be explored before a costly but correct one.

2.3 Iterative Deepening Search (IDS)

IDS repeats **depth-limited DFS** with limits 0, 1, 2, 3, … until the exit is found. It combines the small memory of DFS with the completeness and shallowest-solution property of BFS.

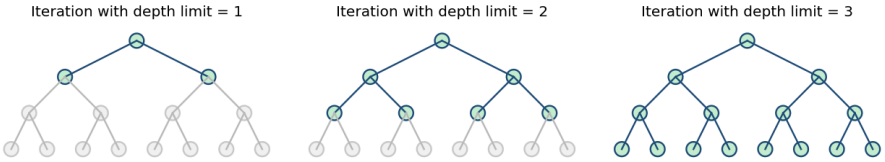


Figure 2: Iterative deepening on a tree: nodes explored in each iteration (green)

- **Use in the maze:** the robot first searches all cells within 1 step, then within 2 steps and so on. Only the current path is stored (plus the visited-cell record), so memory stays small. Depth limits also prevent the endless wandering of plain DFS around loops.
- **Properties:** complete (for finite branching); optimal when step costs are equal (finds the shallowest exit).
- **Time complexity:** O(b^d), where d is the depth of the shallowest exit.
- **Space complexity:** O(b·d), linear in depth, which is the main advantage.
- **Advantages:** very low memory use; no infinite descent; finds the shortest path in unit-cost mazes; good when the depth of the exit is unknown.
- **Disadvantages:** states near the top are regenerated many times, and the algorithm is not optimal for unequal step costs. For b = 10 and d = 5, IDS generates about 123,450 nodes against 111,110 for BFS, an overhead of only about 11%, since most nodes lie at the deepest level. For a physical robot, each new iteration also means travelling back to the start, which costs time and energy, so this repeated work is better done on the robot's internal map than by driving.

2.4 Comparison of UCS and IDS

Criterion	Uniform Cost Search	Iterative Deepening Search
Basic idea	Expand lowest g(n) first (priority queue)	Repeated depth-limited DFS with growing limit
Complete?	Yes (if ε > 0)	Yes
Optimal?	Yes (any positive costs)	Yes only if all step costs are equal
Time	O(b^(1+⌊C*/ε⌋))	O(b^d)
Space	O(b^(1+⌊C*/ε⌋)), very large	O(b·d), very small
Best when	Costs vary and memory is available	Memory is limited and depth is unknown
Weakness	Memory and exploration of cheap dead ends	Repeated work; ignores unequal costs

2.5 Relating the Problem to Types of Intelligent Agents

Agent type	How it behaves in an unknown maze	Suitability
Simple reflex	Acts only on current percept (e.g. “if wall ahead, turn right”); has no memory, so it can walk in circles forever	Poor: partially observable and looping
Model-based	Remembers visited cells and walls and builds a map, so it avoids repeating itself	Necessary for the maze
Goal-based	Has the goal “reach the exit” and plans a sequence of moves with search (BFS/IDS/UCS) on its map	Good: provides the search
Utility-based	Chooses among routes by weighing path cost, time, energy, and risk (a utility function)	Best when costs differ

Effect of agent design on decision-making. The design decides what the robot knows and how it chooses. A memoryless reflex agent decides only on what it senses and can loop forever, whereas a model-based agent decides using its map and therefore avoids loops and dead ends it has already seen. A goal-based agent decides on the sequence of moves that reaches the exit, so it can plan rather than just react; a utility-based agent compares alternatives and so can choose a safe, cheap route over a merely short one. Better agent design means better use of the search algorithm, because the search needs an internal model (the map) and a goal to work with.

2.6 Recommended Combination and Justification

Recommendation: a model-based, goal-based agent (with utility-based cost handling when costs differ) using Iterative Deepening Search as the main strategy, and Uniform Cost Search when step costs are unequal.

- **Model-based:** the maze is unknown and only partially observable, so the robot must record walls, visited cells and openings as it explores.
- **Goal-based:** a clear goal (the exit) lets the agent plan and re-plan with search as the map grows.
- **IDS for unit-cost mazes:** it needs only O(b·d) memory (useful for a small on-board computer), is complete, finds the shortest path, and does not loop. The extra node generation is small.
- **UCS for mazes with different costs:** if some passages are costly (slopes, rough ground, hazards), UCS guarantees the cheapest route; the utility-based part of the agent supplies the cost function.
- **Practical note:** an agent can run IDS or UCS on its internal map and physically walk only the chosen path, reducing the wasted travel of repeated deepening.

Conclusion: UCS is cost-optimal but memory-hungry, while IDS is memory-efficient and finds the shallowest exit; paired with a model-based, goal-based agent (extended with utility when costs vary), they let the robot solve the unknown maze completely and efficiently.